

Simulating Insect Vision for use in Navigation in Dim Light Environments

Nico de Jong¹, Yash Sondhi², Jane Shin¹

¹Department of Mechanical and Aerospace Engineering, University of Florida, Gainesville, FL, USA (email: {n.dejong; jane.shin}@ufl.edu)

¹Department of Physics, Georgia Institute of Technology, Atlanta, GA, USA (email: yashsondhi@gmail.com)

Background

Nocturnal and crepuscular insects possess a remarkable degree of spatial acuity in dim-light environments. This is because their visual systems are designed to maximize light intake with relatively simple setups. Modelling the visual systems of these insects is then of paramount importance to cost-effectively gather and process useful data in the visible light spectrum to eventually use in dim-light navigation.

Methods

I use ray-casting Methods and build off of a renderer constructed by Blayze Millward at the University of Sheffield called CompoundRay[1] to model apposition and superposition compound eyes. The rendered images are then tested for optical flow performance against ray-traced images in a virtual scene constructed through Blender. We use Lukas-Kanade optical flow for this evaluation.

Workflow and Pipeline

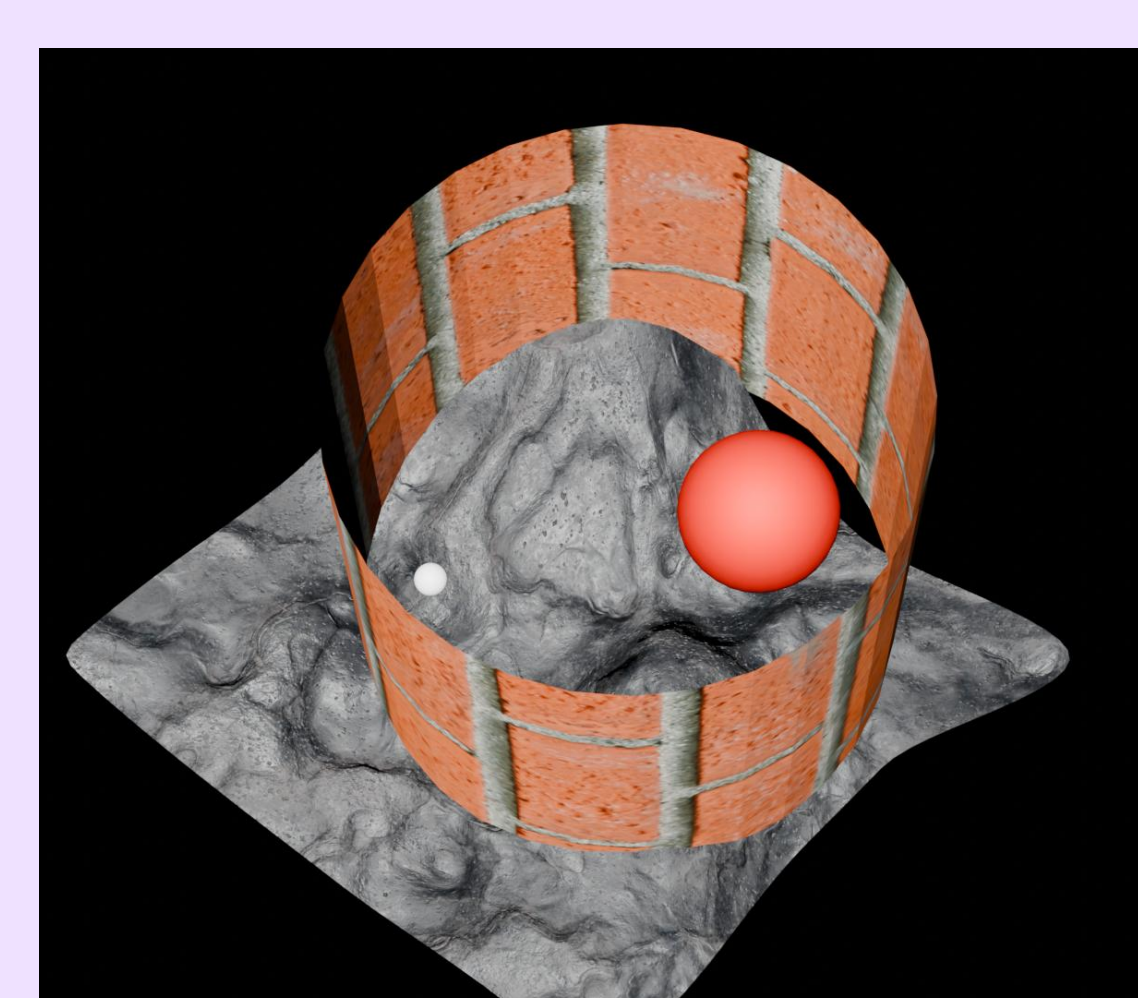


Fig 1.) A virtual Scene is constructed in Blender, with a camera defined and positioned at the white ball, angled towards the orange ball.

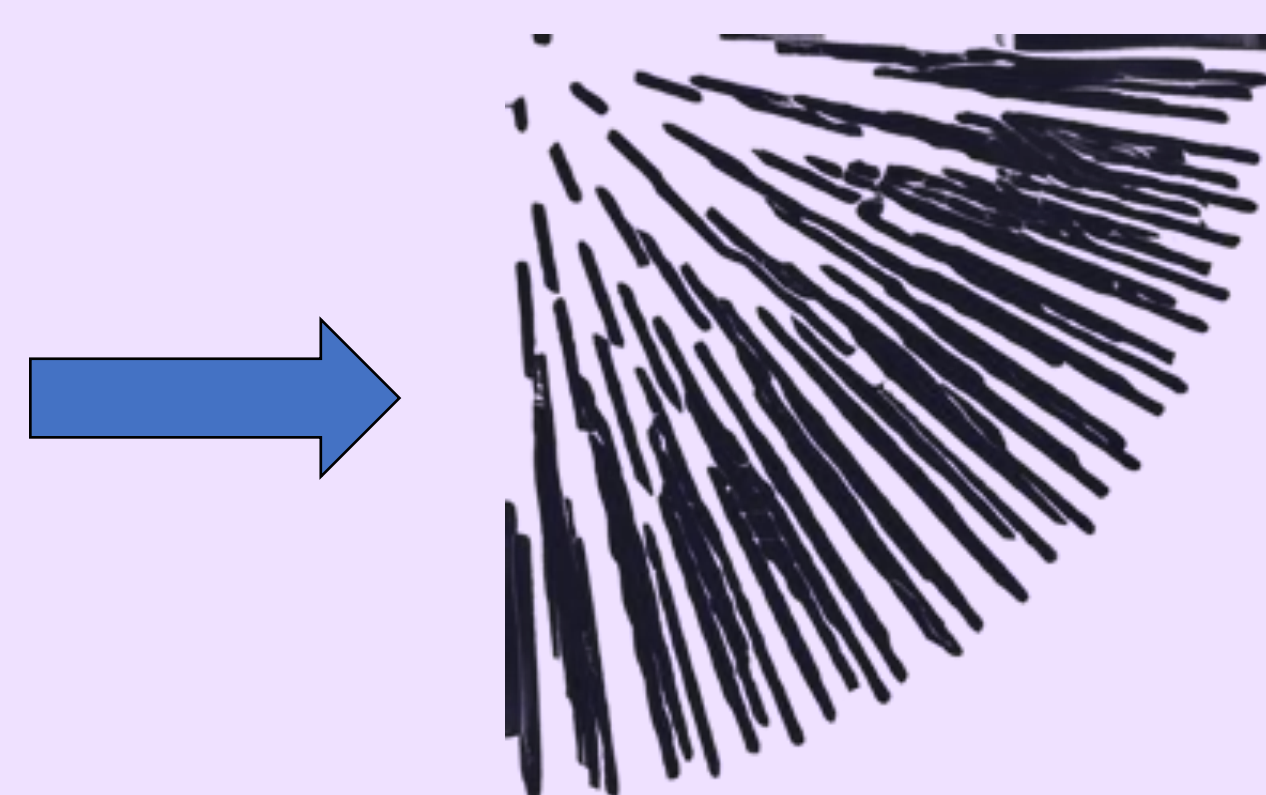


Fig 2.) An eye file, containing an array list of ommatidial origin points, axis directions, focal points and angles of acceptance, is assigned to the camera.

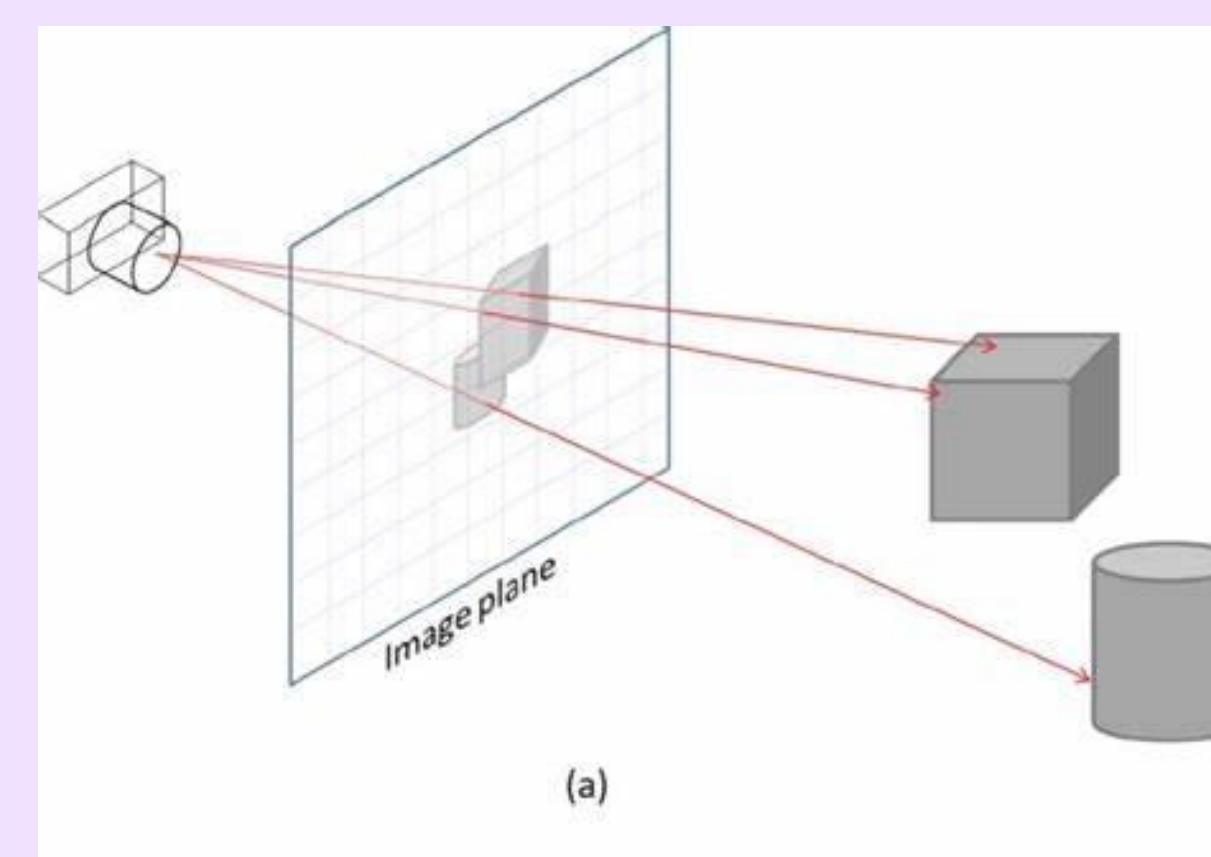


Fig 3.) For each ommatidium ray casting techniques are performed, returning an RGB value for the average area of the facet.



Fig 4.a-d.). Different types of images can then be generated based on eye file input. Displayed from left to right is a.) simulated superposition vision, b.) apposition vision, c.) apposition vision with randomized ommatidia placement, d.) and a simple eye. Superposition was simulated via setting groups of ommatidia to have the same origin point.

Results

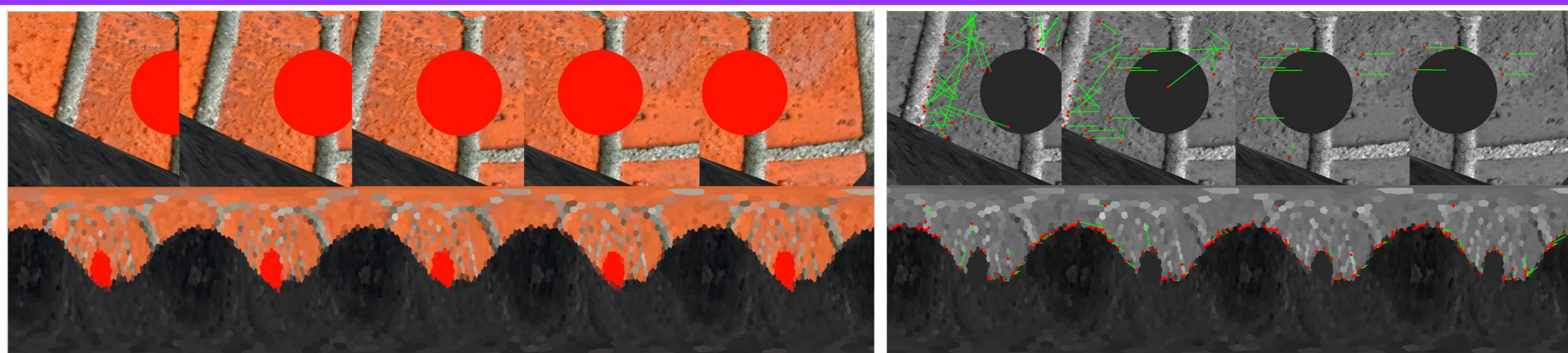


Fig 5.a-j.)

Fig 6.a-h.)

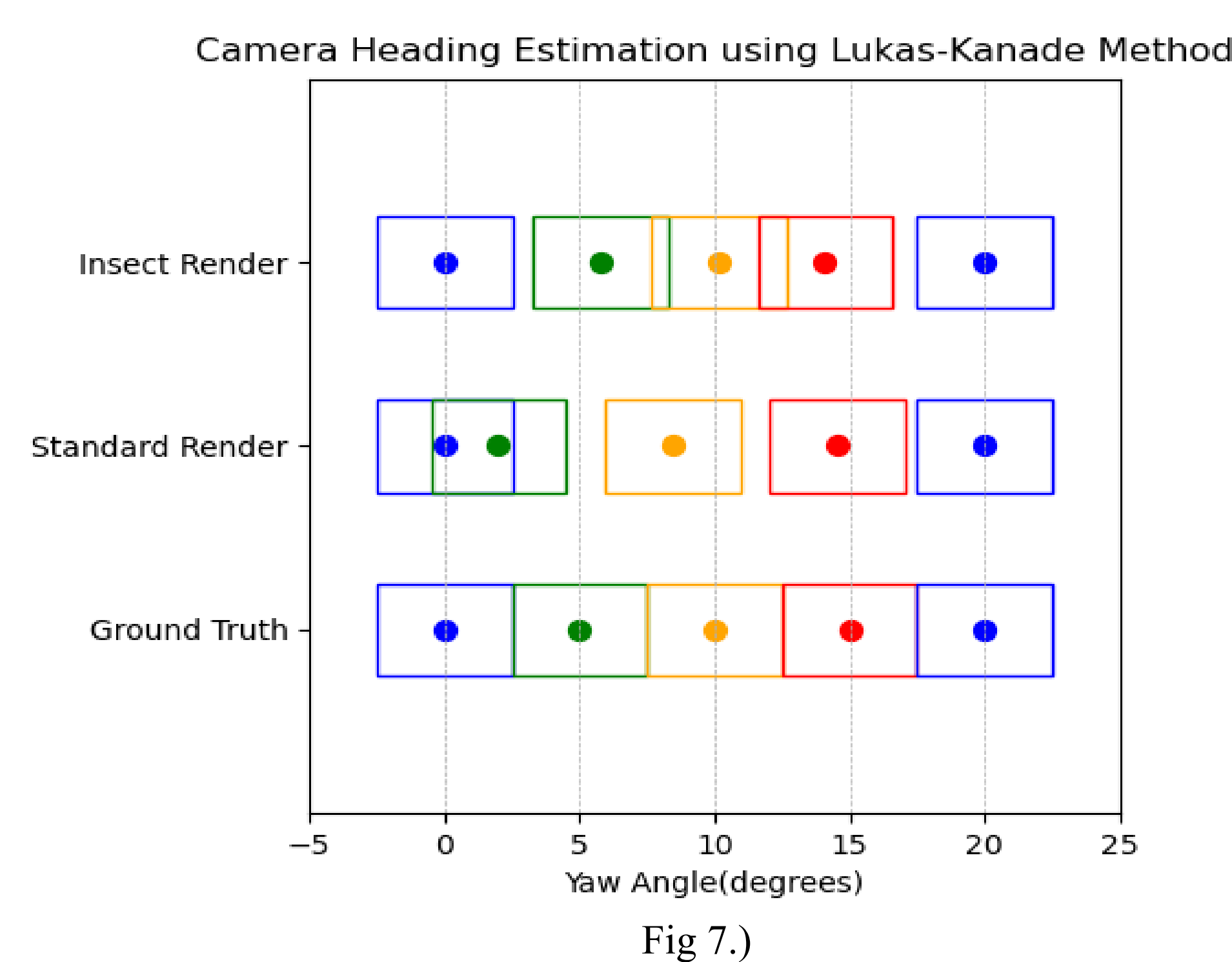


Fig 7.)

Fig 5.a-j.) (With 5.a-e being on the top row) shows the results of two separate renderers as a camera pans 5 degrees horizontally from the left to right, with 5.a. showing a standard rasterized image at 0 degrees yaw, and 5.e. showing the same renderer at 20 degrees yaw. 5.f-j. show the same yaw angles but rendered with Compound Ray.

Fig 6.a-h.) illustrates visual depictions of optical flow results. The red dot pinpoints where the current pixel is, and the end of the green line is where optical flow analysis predicts that pixel will be in the next image.

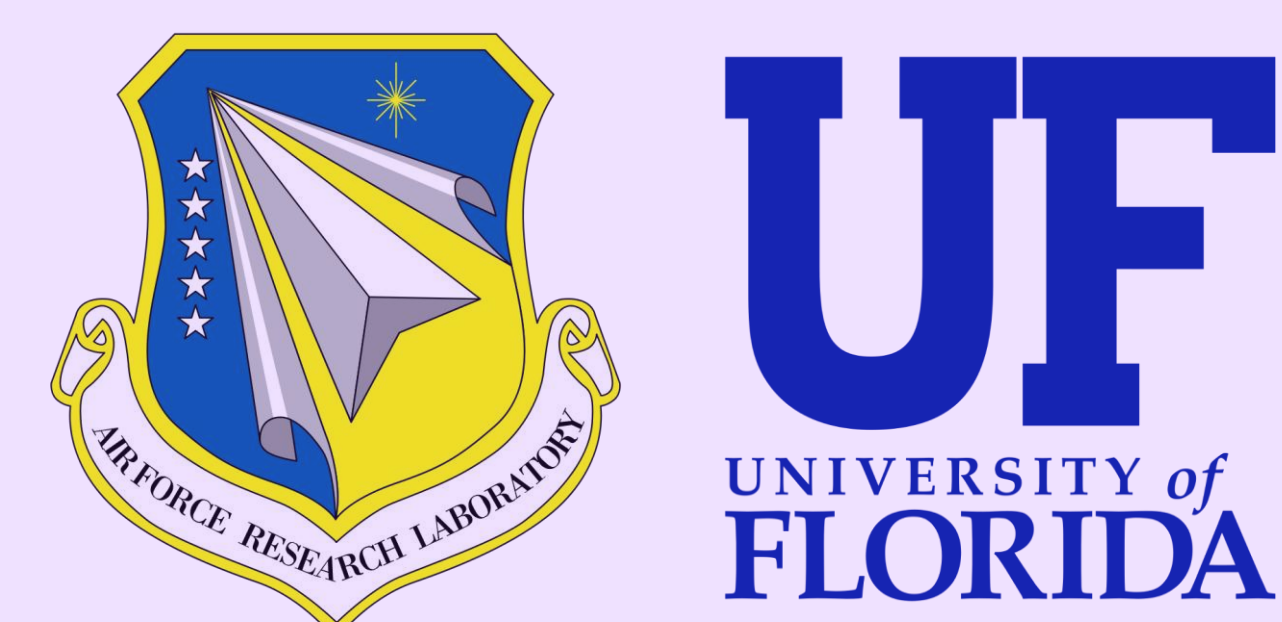
Fig 7.) compares the results of optical flow analysis between the two image sets. The Lukas-Kanade method returns a velocity vector, which is then converted into an estimated yaw angle. The simulated apposition vision image set appears to slightly outperform the simple-eye renderer.

Future Work

The current rendering pipeline does not allow for modelling of polarization, refractive indices, or different lens shapes in compound eyes. Ray-tracing more closely resembles how light reacts in real conditions, but is more computationally expensive. In order to perform real-time rendering realistically and effectively, a compromise must be made between ray-casting and ray-tracing.

Acknowledgments and References

I would like to thank AFRL Scholars, and USRA for providing funding, Blayze Millward for making CompoundRay open source, and Drs. Nathan Lord, Dan Chappell, Ric Wehling, Yash Sondhi, and Jane Shin for their support and mentorship.



[1] Blayze Millward, Steve Maddock, Michael Mangan (2022) CompoundRay, an open-source tool for high-speed and high-fidelity rendering of compound eyes eLife 11:e73893.